Start here: design and run order

What this solution covers

This is a worked learning solution to the attached MIT 8103 portfolio brief. It covers all five portfolio topics using Microsoft SQL Server T-SQL only. Each explicitly created database object has its own PDF. Each folder also contains matching .sql files and a GitHub structure.

Case study: Green Basket Shop is a simulated small shop in Ibadan selling rice, beans and oil. Cashiers record customers and sales, a stock officer manages products, and a manager checks sales totals. The database stores customers, products, order headers, order lines and optional product descriptions.

Scope: One currency (NGN), one shop and one customer per order. One product appears once per order; Quantity records multiple units. Product stock is the current quantity on hand. Payments, delivery and tax are outside this small teaching model.

Software and honest evidence

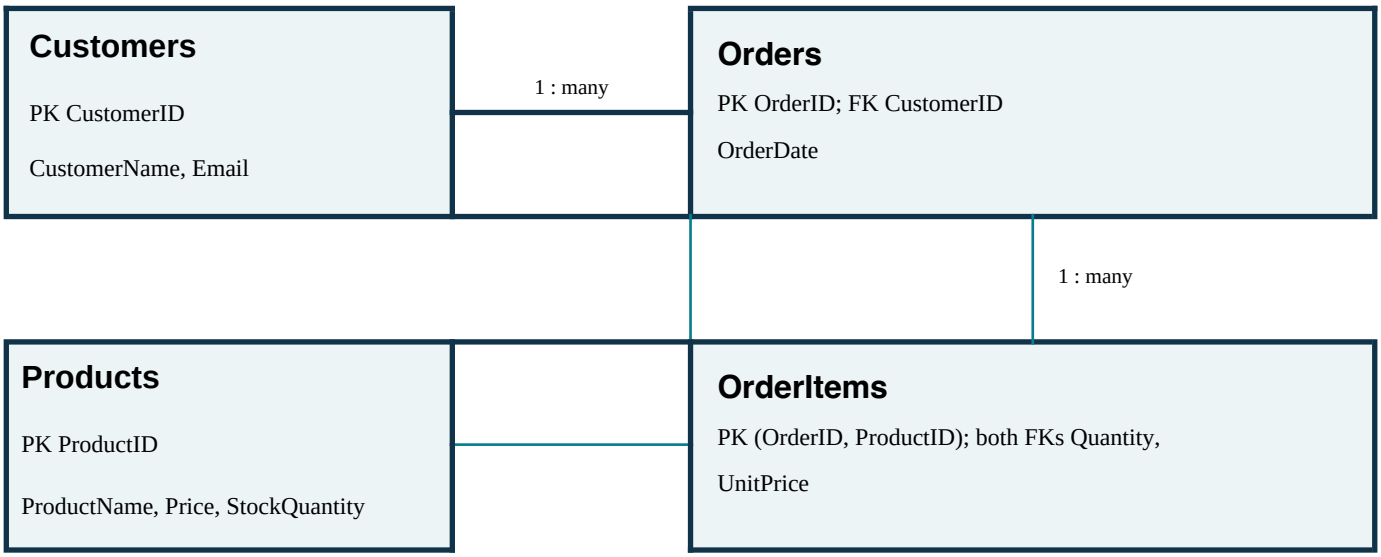
Use Microsoft SQL Server 2014 or later with SQL Server Management Studio (SSMS). GO is an SSMS batch separator, not a statement sent to the SQL Server engine. Open each .sql file in SSMS and run it in the stated order to validate their correctness and accuracy.

NoSQL scope: Portfolio 4 demonstrates a document model stored as JSON through T-SQL. SQL Server remains a relational engine; this is not a separate NoSQL database deployment.

Run order

Step	PDF / action
1	01: create GreenBasketDB. Run once.
2	02, 03, 04, 05: create and populate tables, in this order.
3	06: create and populate ProductDocuments.
4	08: run sales queries and the BEFORE index measurement.
5	07: create the index; return to 08 for the AFTER measurement and cleanup.
6	09: run commit, rollback and two-window concurrency examples.
7	10: run JSON queries; read 11 for the cloud discussion.
8	12: complete the 3-page LMS summary with genuine evidence and your GitHub URL.

Portfolio 1: entity-relationship diagram



Read every 1 : many relationship from the first table to the second. A customer can have zero or many orders; each order has one customer. An order can have zero or many lines while being entered; each line has one order. A product can occur on zero or many lines; each line has one product. A completed sale should contain at least one line, a rule followed by the sale transaction.

ProductDocuments is an optional extension: Products 1 : 0..1 ProductDocuments. Its ProductID is both its primary key and a foreign key. The JSON content contains optional descriptive properties only.

Relational schema

Customers(**CustomerID PK**, CustomerName, Email UNIQUE)
Products(**ProductID PK**, ProductName, Price, StockQuantity)
Orders(**OrderID PK**, CustomerID FK, OrderDate)
OrderItems(**OrderID PK/FK, ProductID PK/FK**, Quantity, UnitPrice)
ProductDocuments(**ProductID PK/FK**, Details)

Normalisation in simple terms

Before normalisation: A single sales sheet might repeat the customer name, email and several product names inside each order.

First normal form (1NF): Each ordinary column holds one value. Separate product lines go in OrderItems, not in a comma-separated list. The JSON extension is discussed separately as a document model.

Second normal form (2NF): OrderItems uses the pair OrderID + ProductID as its key. Quantity and the price charged for that line depend on the whole pair. Customer details belong to Customers and product descriptions belong to Products.

Third normal form (3NF): CustomerName and Email depend on CustomerID. OrderDate and CustomerID depend on OrderID. ProductName, current Price and StockQuantity depend on ProductID. Do not repeat CustomerName in Orders or ProductName in OrderItems.

OrderItems.UnitPrice is the price actually charged, so it may differ from Products.Price after a price change. This is historical sale data, not an accidental duplicate. Order totals are calculated with SUM rather than stored.

Keys and constraints

PRIMARY KEY identifies a row; FOREIGN KEY requires the parent row to exist; NOT NULL requires a value; UNIQUE prevents repeated emails; CHECK prevents invalid prices, quantities and negative stock. These constraints are defined inside the separate table scripts. SQL Server creates supporting indexes for primary and unique keys automatically; they do not need separate manual scripts.

What to submit

Use the .sql files as the practical implementation, the PDFs as explanations, and screenshots and saved .png files as evidence. Actual work is Commit progressively to a private GitHub repository.